

Pulse — "Since You Left" Market Watchlist

End-to-End Implementation Plan for Claude Code

What this is: a build specification for a smart market watchlist whose one job is to answer "*I was away — what meaningfully changed in my watchlist, why does it matter, and what should I look at first?*" This is not a price dashboard. The differentiator is a per-user, race-safe change engine that ranks stocks by how much they genuinely deserve attention.

How to use this document: build in the phases below, in order. Every phase ends with a **Test Gate** — a concrete, runnable check. Do not proceed to the next phase until its gate passes. Each phase is designed to be independently testable.

For the implementer (Claude Code): the four evaluation criteria in §6 are build constraints, not documentation. Before writing each component, check which criterion it earns and make that property *demonstrable* — a passing test, a visible demo moment, or a clearly-commented design choice. Prefer the boring, correct, well-tested version over the clever one; "simplicity without over-engineering" is an explicit grading axis. When a design decision has a more rigorous alternative you're not building (e.g. CDC vs safety-lag), leave a comment naming the tradeoff — that reasoning is itself graded.

0. Ground Rules (read before writing any code)

- No hardcoded market data.** All prices, volumes, and history come from a live provider (Finnhub) or a historical provider (Alpha Vantage / Twelve Data / Stooq), pulled at runtime and stored. The only exception is the deterministic **replay provider**, whose synthetic ticks are generated from a seeded algorithm, not copy-pasted numbers — this is a legitimate test/demo fixture, and it must be clearly labeled as simulated.
- Provider abstraction is mandatory.** All market data flows through one interface (`MarketDataProvider`). Finnhub, the historical seeder, and the replay simulator are all implementations. Nothing else in the codebase calls an HTTP finance endpoint directly.
- The three minimum capabilities must work end-to-end at all times:** (a) create and manage a watchlist, (b) view latest market information, (c) return later and see what changed. Never let a later phase break an earlier one.
- Every number the user sees must carry its provenance:** `as_of` (when the price happened) and `source` . Staleness is surfaced, never hidden.
- The score and every event must be explainable in one plain-English sentence** built only from arithmetic. No ML, no black boxes, no "buy/sell" language ever.
- Write tests alongside each component**, not at the end. The evaluation rewards correctness, edge cases, and resilience — tests are how you prove all three.

1. Data Provider Reality (important — do not skip)

Single live provider: Twelve Data (free Basic plan). One API key covers every data need in this build for US stocks, which keeps the integration simple (a deliberate choice — the "simplicity" criterion rewards it, and fewer providers means fewer failure modes in a 48-hour sprint).

Free-tier limits: 800 API credits/day, 8 requests/minute, ~5,000 data points per request. Most endpoints cost 1 credit per symbol. For a ~12-symbol demo universe this is comfortably within limits.

Demo universe is US symbols (AAPL, MSFT, TSLA, NVDA, AMZN, GOOGL, META, JPM, etc.) — Twelve Data's free tier covers US stocks; other exchanges need paid tiers. The engine is identical regardless of exchange; only the ticker list changes. State this in the README.

Everything comes from one provider:

Purpose	Twelve Data endpoint	Free tier?	Notes
Live quotes (price, day OHLC, prev close, volume)	<code>/quote</code>	✓	Polled during market hours
Historical daily bars (seed volatility, avg vol, 52w, beta)	<code>/time_series?interval=1day</code>	✓	Pulled ONCE at seed time, stored, never re-fetched
Splits / dividends (corporate actions)	<code>/splits</code> , <code>/dividends</code>	✓	Populates <code>adj_factor</code>
Symbol search	<code>/symbol_search</code>	✓	For the add-stock flow
Deterministic test/demo	Replay simulator (in-house)	n/a	Seeded random walk + scripted events. Used when markets are closed and in ALL automated tests

Credit budgeting (state this reasoning in code comments): seed history sequentially at startup (1 credit/symbol, one time); during market hours poll live quotes at an interval that respects 8 req/min and stays under 800/day (e.g. 12 symbols every 2–5 min is fine). Never poll per-user — always the deduplicated union of watchlisted symbols.

Two known Twelve Data caveats to handle: - **Corporate actions can take up to 48h to reflect** in their feed due to verification. Do NOT rely on it catching a split that happens *during* your build — demo split-handling deterministically via the replay simulator instead. - **WebSocket/streaming is paid-only.** REST polling is used, which is fine — a "since you left" product does not need sub-second latency (established design decision). Note in README that a streaming provider could slot in behind the same interface if lower latency were ever required.

Config via environment variables only (never commit keys):

```
TWELVEDATA_API_KEY=...
MARKET_PROVIDER=twelvedata      # twelvedata | replay
```

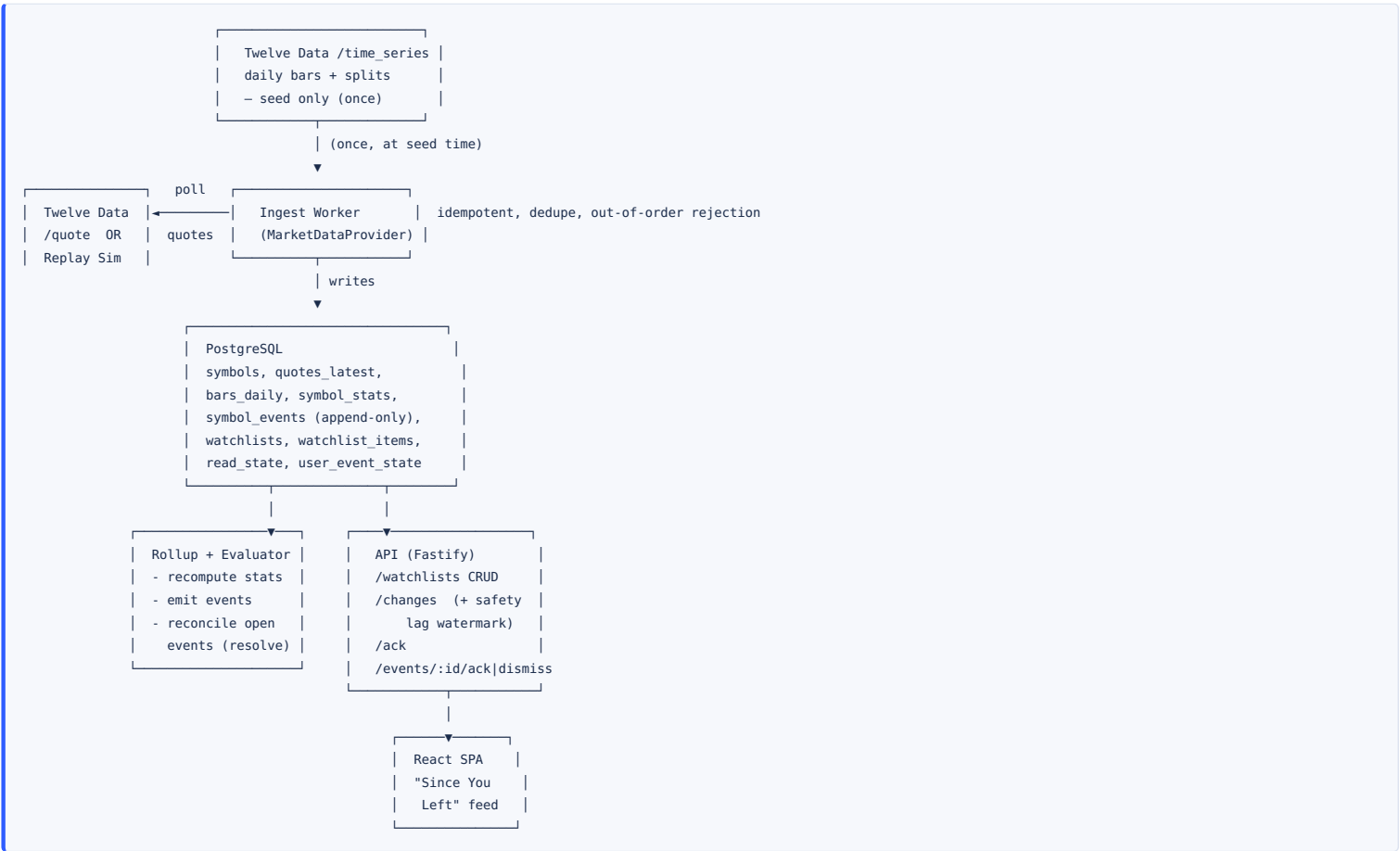
If any signal needs data the free tier does not provide, the fallback is: compute it from the historical bars you seeded, or drop the signal and note it in the README. Do **not** silently fabricate it.

2. Tech Stack (and why)

Layer	Choice	Why
Language	TypeScript end-to-end	One language for API + frontend; strong types catch race/model bugs the evaluation cares about
Backend	Node.js + Fastify (or Express)	Boring, fast, minimal. Fastify has good schema validation
DB	PostgreSQL	Needs real transactions, UNIQUE constraints, ON CONFLICT , sequence types, GREATEST() — all central to the race-safety design. SQLite can substitute for local dev but Postgres is the target
ORM/queries	Prisma (schema + migrations) with raw SQL where the watermark logic needs precision	Prisma for CRUD; raw SQL for the safety-lag watermark query so behaviour is explicit and testable
Scheduler	node-cron (or a simple setInterval loop)	The poller and rollup jobs. No queue infra needed at this scale
Frontend	React + Vite + TypeScript, TanStack Query for server state, plain CSS or Tailwind	TanStack Query handles polling/refetch/staleness cleanly, which maps directly to the product
Tests	Vitest (unit + integration)	Fast, TS-native
Deploy	Single deployable (frontend + API) — Vercel/Render/Railway; Postgres on Neon or Supabase	A single service is preferable to microservices at this scope — over-engineering is explicitly penalised

Deliberately NOT using: Kafka, Redis, Kubernetes, microservices, WebSockets (SSE or simple polling is enough for a "since you left" product — real-time streaming is not the point). Note each of these as a conscious cut in the README.

3. Architecture



Architectural principle to state explicitly: *materiality is computed once per symbol (symbol-centric write); correctness is maintained once per user (user-centric read).* 1M users watching 5,000 unique symbols = 5,000 evaluations, not 50M. The event log is append-only; a user's feed is a cheap indexed join of their watchlist symbols against events above their watermark.

4. Data Model (full schema)

```
-- ----- reference & market data -----
CREATE TABLE symbols (
  symbol      TEXT PRIMARY KEY,
  name        TEXT NOT NULL,
  exchange    TEXT NOT NULL DEFAULT 'US'
);

CREATE TABLE quotes_latest (
  symbol      TEXT PRIMARY KEY REFERENCES symbols(symbol),
  price       NUMERIC NOT NULL,
  day_open    NUMERIC,
  day_high    NUMERIC,
  day_low     NUMERIC,
  prev_close  NUMERIC,
  volume      BIGINT,
  as_of       TIMESTAMPTZ NOT NULL, -- when the price happened (from provider)
  received_at TIMESTAMPTZ NOT NULL, -- when we learned it
);
```

```
source      TEXT NOT NULL
);

CREATE TABLE bars_daily (
  symbol      TEXT NOT NULL REFERENCES symbols(symbol),
  bar_date    DATE NOT NULL,
  open NUMERIC, high NUMERIC, low NUMERIC, close NUMERIC,
  volume BIGINT,
  adj_factor  NUMERIC NOT NULL DEFAULT 1.0, -- corporate-action adjustment
  PRIMARY KEY (symbol, bar_date)
);

-- rolling stats recomputed by the rollup job
CREATE TABLE symbol_stats (
  symbol      TEXT PRIMARY KEY REFERENCES symbols(symbol),
  vol_20d     NUMERIC, -- stddev of daily returns, 20d
  avg_volume_20d NUMERIC,
  high_52w    NUMERIC,
  low_52w     NUMERIC,
  beta        NUMERIC, -- vs market index
  updated_at  TIMESTAMPTZ NOT NULL
);

-- ----- append-only event log -----
CREATE TABLE symbol_events (
  seq          BIGSERIAL PRIMARY KEY, -- the watermark unit
  symbol       TEXT NOT NULL REFERENCES symbols(symbol),
  event_type   TEXT NOT NULL, -- price_move|volume_surge|vol_expansion|
  -- relative_perf|filing|week52_break|gap
  tier         TEXT NOT NULL, -- low|medium|high
  score        NUMERIC NOT NULL, -- 0..100 attention sub-contribution or event score
  reason       TEXT NOT NULL, -- plain-English sentence
  occurred_at  TIMESTAMPTZ NOT NULL,
  resolved_at  TIMESTAMPTZ, -- NULL = still active (Track A)
  trigger_metric TEXT, -- e.g. 'z_score' - for reconciliation
  trigger_threshold NUMERIC, -- e.g. 2.0
  clear_streak INT NOT NULL DEFAULT 0, -- hysteresis counter
  dedupe_key   TEXT UNIQUE NOT NULL -- symbol+type+date-bucket+tier
);
CREATE INDEX ON symbol_events (symbol) WHERE resolved_at IS NULL;

-- ----- watchlists -----
CREATE TABLE users (
  id          UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  created_at  TIMESTAMPTZ NOT NULL DEFAULT now()
);

CREATE TABLE watchlists (
  id          UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  user_id     UUID NOT NULL REFERENCES users(id),
  name        TEXT NOT NULL,
  created_at  TIMESTAMPTZ NOT NULL DEFAULT now()
);

CREATE TABLE watchlist_items (
  id          UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  watchlist_id UUID NOT NULL REFERENCES watchlists(id),
  symbol      TEXT NOT NULL REFERENCES symbols(symbol),
  position    NUMERIC NOT NULL, -- fractional rank, avoids reorder collisions
  added_at    TIMESTAMPTZ NOT NULL DEFAULT now(),
  seed_watermark BIGINT NOT NULL, -- events before this are NOT backfilled to user
  UNIQUE (watchlist_id, symbol) -- duplicate add = no-op
);

-- ----- per-user read state (Track: delivery) -----
CREATE TABLE read_state (
  user_id     UUID NOT NULL,
  watchlist_id UUID NOT NULL REFERENCES watchlists(id),
  watermark   BIGINT NOT NULL DEFAULT 0, -- highest CONFIRMED-delivered seq
  last_seen_at TIMESTAMPTZ, -- coarse "user opened the list"
  PRIMARY KEY (user_id, watchlist_id)
);

-- ----- per-user, per-event acknowledgement (Track B) -----
CREATE TABLE user_event_state (
  user_id     UUID NOT NULL,
  event_id    BIGINT NOT NULL REFERENCES symbol_events(seq),
  acknowledged_at TIMESTAMPTZ,
  dismissed_at TIMESTAMPTZ,
  PRIMARY KEY (user_id, event_id)
);
```

5. Build Phases

Phase 1 — Skeleton + Provider Interface

Goal: the abstraction and a running server, before any real data.

Build: - `MarketDataProvider` interface: `typescript interface Quote { symbol: string; price: number; dayOpen?: number; dayHigh?: number; dayLow?: number; prevClose?: number; volume?: number; asOf: Date; source: string; } interface MarketDataProvider { getQuotes(symbols: string[]): Promise<Quote[]>; } - ReplayProvider implements MarketDataProvider — deterministic seeded random walk; accepts a script of injected events (gap, spike, split, feed-outage, duplicate tick, out-of-order tick). - TwelveDataProvider implements MarketDataProvider — calls /quote, maps fields, sets source='twelvedata', asOf from the quote timestamp. Respects rate limits (8 req/min); normalizes symbols to plain US tickers. A separate TwelveDataHistorical module wraps /time_series and /splits for the one-time seed (Phase 3). - Fastify server with a /health endpoint reporting provider freshness + error rate + remaining daily credit budget (track credits spent). - DB migrations for all tables in $4.`

Test Gate 1: - Unit: `ReplayProvider` returns deterministic quotes for a fixed seed (same seed → same sequence). - Unit: `TwelveDataProvider` maps a sample `/quote` JSON payload to a `Quote` correctly (mock the HTTP call — no live dependency in tests). - Integration: `GET /health` returns 200 with provider status. - Migrations apply cleanly to a fresh DB.

Phase 2 — Ingestion (idempotent, ordered, deduped)

Goal: get quotes into `quotes_latest` safely.

Build: - Poller loop driven by the **union of all watchlisted symbols**, deduplicated (never per-user). Cadence adaptive: - market open → 10s; pre/post market → few minutes; weekend/holiday → stop, serve last close. - Build `isMarketOpen()` with a US market-hours + holiday calendar. - On each quote: - **Out-of-order rejection:** if incoming `asOf <= stored_as_of` for that symbol, discard. - **Idempotent write:** upsert `quotes_latest` on `symbol`. - On provider error / 429 → exponential backoff, mark data stale (do not crash, do not overwrite good data with nulls).

Test Gate 2: - Out-of-order quote (older `asOf`) is rejected; newer is accepted. - Duplicate quote (same `asOf`) does not double-write or error. - Simulated 429 triggers backoff and marks staleness, does not lose the last good price. - Poller only requests the union of watched symbols (assert no request for an unwatched symbol).

Phase 3 — Historical Seed + Rollup Stats

Goal: compute the baselines every signal depends on.

Build: - `TwelveDataHistorical` — fetch ~1 year of daily bars per symbol **once** via `/time_series?interval=1day`, store in `bars_daily`. Seed sequentially respecting 8 req/min, cache to DB, never re-fetch what you already have. - **Corporate-action adjustment:** pull `/splits` and populate `adj_factor` so cross-split comparisons don't fabricate a crash. At minimum handle splits. (Note: live splits may lag up to 48h in the feed — the deterministic split test below does not depend on the live feed.) - Rollup job (`node-cron`, every N minutes while market open) computing into `symbol_stats` : - `vol_20d` = stddev of last 20 daily returns (floored at a minimum to avoid divide-by-zero on illiquid names) - `avg_volume_20d` - `high_52w`, `low_52w` - `beta` vs a market index (use SPY or a broad ETF as the index proxy for US symbols)

Test Gate 3: - Seeded bars produce correct `vol_20d` / `avg_volume_20d` on a known fixture (hand-computed expected values). - A split in the fixture is adjusted: no fake —50% gap appears across the split date. - Illiquid fixture (near-zero volatility) does not produce infinite/NaN stats — floor applies.

Phase 4 — Signal Computation

Goal: turn stats + quotes into individual signals. Each signal is pure and unit-testable.

Implement each as a pure function `(quote, stats, sinceWindow) -> Signal | null` :

Signal	Rule
Price move	residual = actualReturn – beta×indexReturn; <code>z = residual / vol_20d</code> ; fires if z ≥ threshold. Window = since last visit.
Volatility expansion	recent short-window stddev ÷ <code>vol_20d</code> ; fires if ratio high
Volume anomaly	<code>volume / avg_volume_20d</code> ; band into normal/elevated/significant/exceptional. Never claim a cause
Relative performance	stock return – benchmark(index/sector) return, in percentage points
52-week break	price ≥ <code>high_52w</code> or ≤ <code>low_52w</code>
Gap	(day_open – prev_close) / prev_close, scaled by vol
Filing / event	Twelve Data <code>/earnings</code> + <code>/splits</code> + <code>/dividends</code> give structured, dated corporate events (earnings date, split, dividend record date) — keyword/type-match into: earnings / corporate-action categories. No NLP, no free-text news. If richer filing categories are wanted, seed a small labeled set for the demo and mark as simulated in README

Each `Signal` carries `{ type, zscore|ratio, subScore (0..100), message, tier }`.

Sub-score: `subScore = min(100, (z / MAX_Z) * 100)` with `MAX_Z ≈ 4`. Binary signals (filing, 52w break) fire at a fixed sub-score (e.g. 70).

Test Gate 4 (this is a high-value gate for the evaluation): - Calm large-cap 2% move → low/no price signal; volatile small-cap same 2% → also correctly scored relative to its own vol. - **Market-wide selloff:** stock moves exactly at its beta with the index → residual ≈ 0 → **no price signal** (proves market-adjustment). - Split date → no fabricated price signal (reuses Phase 3 adjustment). - Illiquid → floored vol, no infinite z. - Volume 3× → "exceptional" band, message contains no causal claim. - Each signal's `message` is a complete plain-English sentence.

Phase 5 — Attention Score (the corrected model)

Goal: combine a stock's fired signals into one ranked 0–100 score.

Do NOT use a naive weighted sum. Use a weighted **average** of sub-scores (auto-caps at 100, no artificial clamp), then an **agreement multiplier**:

```
const WEIGHTS = {
  price: 0.30, volatility: 0.15, volume: 0.20,
  relative: 0.15, event: 0.20,      // event covers filing / 52w / gap
};

function attentionScore(signals: Signal[]): { score: number; tier: string } {
  let base = 0;
  for (const s of signals) base += (WEIGHTS[s.category] ?? 0) * s.subScore;

  const fired = signals.filter(s => s.subScore > 40).length;
  let mult = 1.0;
  if (fired >= 3) mult = 1.15;
  else if (fired === 2) mult = 1.05;

  const score = Math.min(100, base * mult);
  const tier = score >= 70 ? 'high' : score >= 40 ? 'medium' : 'low';
  return { score, tier };
}
```

Rationale to put in README: *averaging keeps the score meaningful on its own scale without an arbitrary clamp; a separate agreement multiplier rewards genuine corroboration without letting several mediocre signals fake a strong one.*

The plain-English **reason** for a stock = its top 2-3 contributing signals' messages, concatenated. Nothing invented.

Test Gate 5: - All-zero signals → score ≈ 0, tier low. - One strong signal only → moderate-high, no multiplier applied. - Three moderate signals (each >40) → multiplier lifts into "high" though none alone would. - Score never exceeds 100. - Generated reason references only signals that actually fired.

Phase 6 — Event Emission + Dedupe + Resolution (Tracks A & B)

Goal: persist events immutably, never double-alert, auto-resolve when conditions clear.

Build: - Evaluator emits a `symbol_events` row when a signal (or combined score) crosses a tier, with: - `dedupe_key = symbol + ':' + type + ':' + dateBucket + ':' + tier` - `INSERT ... ON CONFLICT (dedupe_key) DO NOTHING` (idempotent — retries harmless) - `trigger_metric` + `trigger_threshold` stored for reconciliation - **Track A — resolution via recheck-on-rollup (chosen over time-decay):** - Every rollup pass, for each symbol with open events (`resolved_at IS NULL`), re-evaluate the stored `trigger_metric` against fresh stats. - **Hysteresis:** require the condition to be clear for **2 consecutive** passes before resolving. Increment `clear_streak` when clear, reset to 0 when it holds; resolve when `clear_streak >= 2` . - Keep `updateStats(symbol)` and `reconcileOpenEvents(symbol)` as separate functions. - **Escalation rule:** a worse move / fresh occurrence gets a **new** dedupe_key (different tier or date-bucket) → new row, clean state, inherits no ack. Never mutate or reopen an old row.

Test Gate 6: - Same event detected twice in one pass → one row (dedupe). - Condition persists across passes → event stays open. - Condition clears once then holds again → NOT resolved (hysteresis). - Condition clears two passes running → resolved. - Aacked medium event then escalates to high → new unacked row appears. - Symbol with no open events → reconciliation does no wasted work.

Phase 7 — Watermark `/changes` + `/ack` (race-safety core)

Goal: the returning-user read, correct under concurrency. **This is the invariant to demo.**

The safety-lag rule: never advance or read the watermark up to the raw `MAX(seq)` . A `BIGSERIAL` assigns numbers before commit, so a slow transaction holding a lower seq can commit *after* a faster higher one — reading `MAX(seq)` would skip the lower row forever. Read only up to a horizon that lags the newest seq, giving in-flight writes time to land.

Implement the horizon as: `safe_seq = MAX(seq) among rows whose occurred_at < now() - INTERVAL '2 seconds'` (or `MAX(seq) - N`). Justify the choice in README (simple single-row inserts, no long transactions, so a 2s lag is an ample margin; the theoretically-airtight alternative is commit-order/CDC, deliberately not built).

- `GET /watchlists/:id/changes` : 1. Compute `safe_seq` (lagged horizon). 2. Fetch events where `seq > read_state.watermark AND seq <= safe_seq` , joined to the watchlist's current items, filtered by:
 - `resolved_at IS NULL` (Track A)
 - no `user_event_state` ack/dismiss for this user (Track B)
 - `seq > watchlist_items.seed_watermark` (no backfill for freshly-added symbols)3. Rank by attention score desc. Return the events **and** the `safe_seq` as `cursor` . 4. **Do NOT advance the watermark here.**
- `POST /watchlists/:id/ack { cursor }` :
- `UPDATE read_state SET watermark = GREATEST(watermark, :cursor)` — confirm-then-advance; `GREATEST` prevents a stale device rewinding it.
- `POST /events/:id/acknowledge` and `/dismiss` : upsert `user_event_state` (idempotent via PK).

Test Gate 7 (the invariant — if you demo one thing, demo this): - **No lost event:** write an event *between* a `/changes` read and its `/ack` ; assert it appears on the next `/changes` . - **Safety-lag gap:** simulate a slow (uncommitted) low-seq insert + a fast committed high-seq insert; assert the low-seq event is not skipped once it commits. - **Double-ack** is a no-op. - **Stale-device ack** (lower cursor) does not rewind the watermark (`GREATEST`) . - **Freshly-added symbol** does not dump backfilled history (`seed_watermark`). - Two devices, one watchlist: events delivered exactly once, no loss.

Phase 8 — Watchlist CRUD + Concurrency

Goal: the first two required capabilities, race-safe.

Build: - Create / rename / delete watchlist; multiple per user. - Add / remove / reorder symbols; symbol search (Twelve Data `/symbol_search`) . - Duplicate add → no-op via `UNIQUE(watchlist_id, symbol)` + upsert. - Reorder: send the single moved item + its new **fractional** position (e.g. 1.5 between 1 and 2), never the whole array — avoids lost-update on concurrent reorder. - New symbol seeds `seed_watermark = current safe_seq` so no backfill.

Test Gate 8: - Duplicate add from two "devices" → one row, no error. - Concurrent reorder of two different items → both survive (no lost update). - Delete a symbol that has events → its events vanish from `/changes` (membership filtered at read time).

Phase 9 — Frontend ("Since You Left")

Goal: the returning-user experience, staleness-honest.

Build: - **Briefing landing:** header "N things need your attention since {last seen, in market terms}". "Last checked: yesterday 3:45 PM", not "18h ago", snapped to market boundaries. - **Priority feed:** ranked cards, each showing what changed / compared with what / why it matters / how fresh / source. Tier colour: green up, red down, amber unusual, blue new info, grey stale/unchanged — plus a text label + icon (never colour alone, for accessibility). - **Quiet section:** "No meaningful change: AAPL, MSFT..."

collapsed. This is a feature. - **Detail drawer:** per-signal breakdown + freshness per data group (price 2m ago, filing 18m ago — never one generic "updated"). "Mark reviewed" → `/acknowledge`. - TanStack Query polls `/changes`; on render of the feed, call `/ack` with the returned cursor. - **All states built:** empty, loading, stale banner, error, market-closed.

Test Gate 9: - First-time user (empty), returning user with changes, returning user with no changes all render correctly. - Stale data shows the banner; market-closed shows the closed state with last-close framing. - Marking an item reviewed removes it from the priority feed and it stays gone on refresh.

Phase 10 — Resilience, Demo Mode, Docs

Goal: prove the edge cases and package the narrative.

Build: - **Demo/replay mode** (`MARKET_PROVIDER=replay`): a scripted scenario — a gap, a volume spike, a 52-week break, a split, a feed outage — triggerable on demand so the demo doesn't depend on live market hours or the Twelve Data credit budget. - **Feed-kill demo:** stop the provider mid-run; UI degrades to "last known price, N min old, reconnecting"; recovers automatically. - **Two-device demo:** two browser windows, one watchlist, show no lost events and convergent edits. - **Split demo:** trigger a split in replay; show naive path would scream —50%, your path stays silent. - README with: architecture diagram, the "compute once per symbol / correct once per user" principle, the safety-lag decision and the CDC alternative you rejected, the scope-cut table (auth, trades, charts, sentiment, WebSockets, microservices — each with a one-line reason), and a "what I'd do at 100x" section.

Test Gate 10: - Full replay scenario runs end-to-end deterministically from a fixed seed. - Feed-kill → graceful degradation → auto-recovery, all visible. - `npm test` green across all phases.

6. Evaluation Criteria — Enforceable Contract

This project is judged on four dimensions. They are not a grading afterthought — they are build constraints. Every phase must actively earn them, and the README must point to where each is demonstrated. Treat this section as acceptance criteria.

6.1 Engineering Depth — *architecture, correctness, reliability, scalability*

Must be demonstrable, not just claimed: - **Scalability by construction:** materiality computed once per symbol, read per user. The poller hits the deduplicated union of watchlisted symbols, never per-user. A reviewer must be able to point at the code and see that 1M users × 5,000 symbols = 5,000 evaluations, not 50M. - **Correctness under concurrency:** the safety-lag watermark (Phase 7) with confirm-then-advance `/changes` → `/ack`, `GREATEST()` on advance. This is the headline correctness property. - **Reliability:** provider failures degrade gracefully (Phase 2 backoff, Phase 10 feed-kill), never crash, never overwrite good data with nulls. - **Clean layering:** provider abstraction means the finance vendor is swappable without touching business logic.

6.2 Product & Problem Interpretation — *understanding beyond the obvious brief*

The brief says "don't build the obvious watchlist." Prove you didn't: - The landing screen answers "what changed since I left," not "here are prices." Ranked by attention, not alphabetical. - Absence windows in market terms ("since Friday's close"), not raw hours. - The quiet section ("nothing meaningful changed in these 9") is a deliberate feature, not an empty state. - The corrected attention score (weighted average + agreement multiplier) — rewards corroboration, resists noise. The *why* is explainable in one sentence per stock. - Never uses buy/sell language — framed as an information tool (§8). This is a product-judgment signal, not just compliance.

6.3 Edge Cases & Resilience — *failures, race conditions, integrity, unreliable dependencies*

Each of these must have a **passing test** and, where possible, a **visible demo moment**: - Out-of-order quote rejection; duplicate-ingest idempotency (Phase 2). - Split not rendered as a —50% crash (Phase 3–4) — demo it in replay. - Illiquid-symbol divide-by-zero floored (Phase 4). - Market-wide selloff produces no false alert via market-adjustment (Phase 4). - Watermark loses no event written mid-request; safety-lag covers the seq-visibility gap (Phase 7). - Resolution hysteresis prevents flapping (Phase 6). - Concurrent watchlist edits: no lost update, no duplicate row (Phase 8). - Feed outage → visible degradation → auto-recovery (Phase 10).

6.4 Code Quality & Simplicity — *maintainability without over-engineering*

- Signal functions are **pure** (`quote, stats, window`) -> `Signal | null` — trivially testable, no hidden state.
- `updateStats()` and `reconcileOpenEvents()` kept separate (Phase 6) for independent testing.
- One service + one Postgres, no Kafka/Redis/K8s/microservices. Every dependency justified in README.
- An explicit **scope-cut table** in the README (auth, trading, charts, sentiment, streaming, microservices — each with a one-line reason). A stated cut reads as judgment; an unstated gap reads as failure.
- Every test gate in §5 is wired into `npm test` and green.

6.5 What the README must contain (the graded narrative)

- Architecture diagram + the "compute once per symbol / correct once per user" principle.
- "What counts as a meaningful change" — the signal set and the attention-score design, with the split/market-adjustment worked example.
- The safety-lag decision **and** the commit-order/CDC alternative you deliberately rejected, with reasoning.
- The scope-cut table.
- A "what breaks first at 100x and what I'd do" section.
- Provider note: Twelve Data chosen as sole provider for simplicity; abstraction allows swapping/adding a streaming provider.
- Honest data caveat: US symbols on free tier; live splits may lag, so split-handling is demoed via the deterministic simulator.

7. The Three Required Capabilities — always working

- Create and manage a watchlist** → Phase 8.
- View latest market information** → Phase 2 + Phase 9 (with staleness).
- Return later and see what changed** → Phases 5–7 + Phase 9 (the differentiator).

Never merge a phase that breaks one of these three. Run their happy-path tests after every phase.

8. Language & Compliance Discipline (non-negotiable)

Never output "Buy / Sell / Strong Buy / target price / institutional buying". Use "High/Medium/Low attention", "Review recommended", "Unusual activity", "Positive price-volume alignment". Frame the product as an information & monitoring tool, not investment advice. State this as a deliberate boundary in the README.